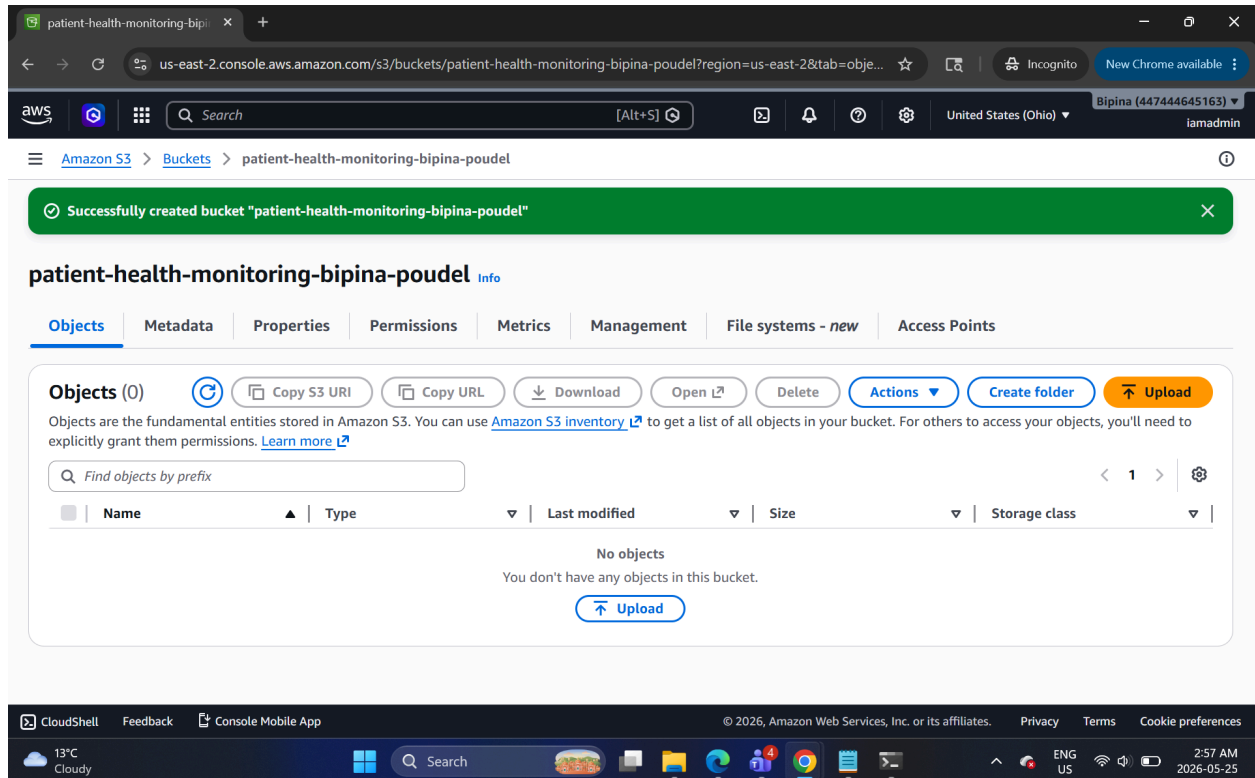


# 1. Project setup

Create an AWS project called **Patient Health Monitoring Data Pipeline**. The goal is to ingest, validate, clean, transform, and store patient monitoring data securely.

## 2. Create S3 bucket

Created an S3 bucket: patient-health-monitoring-bipina-poudel



Inside it, created these folders:

```
raw/vitals/  
raw/patients/  
raw/devices/  
processed/vitals/  
processed/patients/  
processed/reports/  
rejected/
```

patient-health-monitoring-bipi

us-east-2.console.aws.amazon.com/s3/buckets/patient-health-monitoring-bipina-poudel?region=us-east-2&prefix=ra...

Incognito

New Chrome available

aws

Search

[Alt+S]

United States (Ohio)

Bipina (447444645163)

iamadmin

Amazon S3

Buckets

patient-health-monitoring-bipina-poudel

raw/

Successfully created folder "devices".

raw/

Copy S3 URI

Objects

Properties

Objects (3)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

| <input type="checkbox"/> | Name      | Type   | Last modified | Size | Storage class |
|--------------------------|-----------|--------|---------------|------|---------------|
| <input type="checkbox"/> | devices/  | Folder | -             | -    | -             |
| <input type="checkbox"/> | patients/ | Folder | -             | -    | -             |
| <input type="checkbox"/> | vital/    | Folder | -             | -    | -             |

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

13°C

Cloudy

Search

ENG

US

3:04 AM

2026-05-25

patient-health-monitoring-bipi

us-east-2.console.aws.amazon.com/s3/buckets/patient-health-monitoring-bipina-poudel?region=us-east-2&prefix=pr...

Incognito

New Chrome available

aws

Search

[Alt+S]

United States (Ohio)

Bipina (447444645163)

iamadmin

Amazon S3

Buckets

patient-health-monitoring-bipina-poudel

processed/

Successfully created folder "devices".

processed/

Copy S3 URI

Objects

Properties

Objects (3)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

| <input type="checkbox"/> | Name      | Type   | Last modified | Size | Storage class |
|--------------------------|-----------|--------|---------------|------|---------------|
| <input type="checkbox"/> | devices/  | Folder | -             | -    | -             |
| <input type="checkbox"/> | patients/ | Folder | -             | -    | -             |
| <input type="checkbox"/> | vital/    | Folder | -             | -    | -             |

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

13°C

Cloudy

Search

ENG

US

3:05 AM

2026-05-25

patient-health-monitoring-bipina-poudel info

Objects (3)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

|                          | Name                       | Type   | Last modified | Size | Storage class |
|--------------------------|----------------------------|--------|---------------|------|---------------|
| <input type="checkbox"/> | <a href="#">processed/</a> | Folder | -             | -    | -             |
| <input type="checkbox"/> | <a href="#">raw/</a>       | Folder | -             | -    | -             |
| <input type="checkbox"/> | <a href="#">rejected/</a>  | Folder | -             | -    | -             |

### 3. Created and uploaded CSV files

Created csv files and uploaded them in proper destinations.

vital/

Copy S3 URI

Objects (1)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

|                          | Name                               | Type | Last modified                      | Size    | Storage class |
|--------------------------|------------------------------------|------|------------------------------------|---------|---------------|
| <input type="checkbox"/> | <a href="#">patient_vitals.csv</a> | csv  | May 25, 2026, 03:15:14 (UTC-04:00) | 339.0 B | Standard      |

Amazon S3 > Buckets > patient-health-monitoring-bipina-poudel > raw/ > patients/

## patients/

Copy S3 URI

Objects Properties

Objects (1)



Copy S3 URI

Copy URL

Download

Open i

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

< 1 > ⚙

| <input type="checkbox"/> | Name                                | Type | Last modified                      | Size    | Storage class |
|--------------------------|-------------------------------------|------|------------------------------------|---------|---------------|
| <input type="checkbox"/> | <a href="#">patient_details.csv</a> | csv  | May 25, 2026, 03:17:20 (UTC-04:00) | 260.0 B | Standard      |

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

13°C  
Cloudy



Search



ENG  
US

3:17 AM  
2026-05-25

Amazon S3 > Buckets > patient-health-monitoring-bipina-poudel > raw/ > devices/

## devices/

Copy S3 URI

Objects Properties

Objects (1)



Copy S3 URI

Copy URL

Download

Open i

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

< 1 > ⚙

| <input type="checkbox"/> | Name                            | Type | Last modified                      | Size    | Storage class |
|--------------------------|---------------------------------|------|------------------------------------|---------|---------------|
| <input type="checkbox"/> | <a href="#">device_logs.csv</a> | csv  | May 25, 2026, 03:18:23 (UTC-04:00) | 201.0 B | Standard      |

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

13°C  
Cloudy



Search



ENG  
US

3:18 AM  
2026-05-25

## 4. Created IAM roles

### Glue IAM Role and Lambda IAM Role

Roles | IAM | Global

us-east-1.console.aws.amazon.com/iam/home?region=us-east-2#/roles

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

Roles

Policies

IAM users

IAM user groups

Identity providers

Account settings

Root access management

Temporary delegation requests

Access reports

Access Analyzer

Resource analysis

Unused access

Role Lambda-IAM-role created. View role

Roles (15) Info

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Search

| <input type="checkbox"/> | Role name                                                            | Trusted entities                 | Last activity |
|--------------------------|----------------------------------------------------------------------|----------------------------------|---------------|
| <input type="checkbox"/> | <a href="#">databricks-s3-ingest-90716-db_s3_iam</a>                 | Account: 447444645163, and 1 mor | 21 days ago   |
| <input type="checkbox"/> | <a href="#">databricks-s3-ingest-90716-functionRole-RIGTLDMtrNkZ</a> | AWS Service: lambda              | 22 days ago   |
| <input type="checkbox"/> | <a href="#">Glue-IAM-role</a>                                        | AWS Service: glue                | -             |
| <input type="checkbox"/> | <a href="#">glue-role</a>                                            | AWS Service: glue                | 10 days ago   |
| <input type="checkbox"/> | <a href="#">lambda_trigger-role-q6x3gqb9</a>                         | AWS Service: lambda              | -             |
| <input type="checkbox"/> | <a href="#">Lambda-IAM-role</a>                                      | AWS Service: lambda              | -             |
| <input type="checkbox"/> | <a href="#">lambda-role</a>                                          | AWS Service: lambda              | 13 days ago   |

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

13°C Cloudy

Search

ENG US

3:22 AM 2026-05-25

## 5. Created Glue database: healthcare\_db

Databases - AWS Glue Console

us-east-2.console.aws.amazon.com/glue/home?region=us-east-2#/v2/data-catalog/databases/view/healthcare\_db?ca...

AWS Glue

Getting started

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows (orchestration)

Zero-ETL integrations New

Data Catalog

Catalogs

Databases

Tables

Stream schema registries

Schemas

Connections

Crawlers

Classifiers

Catalog settings

Data Integration and ETL

healthcare\_db

Last updated (UTC) May 25, 2026 at 07:27:32 Edit Delete

Database properties

| Name          | Description | Location | Created on (UTC)         |
|---------------|-------------|----------|--------------------------|
| healthcare_db | -           | -        | May 25, 2026 at 07:26:45 |

Tables (0)

Last updated (UTC) May 25, 2026 at 07:27:34 Delete Add tables using crawler Add table

View and manage all available tables.

Filter tables

| Name                | Database | Location | Classific... | Depreca... | View data | Data quality | Column |
|---------------------|----------|----------|--------------|------------|-----------|--------------|--------|
| No available tables |          |          |              |            |           |              |        |

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

13°C Cloudy

Search

ENG US

3:27 AM 2026-05-25

## 6. Created Glue crawler: healthcare-crawler

**One crawler successfully created**  
The following crawler is now created: "healthcare-crawler"

healthcare-crawler  
Last updated (UTC) May 25, 2026 at 07:36:04 [Run crawler](#) [Edit](#) [Delete](#)

**Crawler properties**

|                                     |                                                  |                                          |                          |
|-------------------------------------|--------------------------------------------------|------------------------------------------|--------------------------|
| <b>Name</b><br>healthcare-crawler   | <b>IAM role</b><br><a href="#">Glue-IAM-role</a> | <b>Database</b><br>healthcare_db         | <b>State</b><br>READY    |
| <b>Description</b><br>-             | <b>Security configuration</b><br>-               | <b>Lake Formation configuration</b><br>- | <b>Table prefix</b><br>- |
| <b>Maximum table threshold</b><br>- |                                                  |                                          |                          |

**Advanced settings**

[Crawler runs](#) | [Schedule](#) | [Data sources](#) | [Classifiers](#) | [Tags](#)

**Crawler runs (0)**  
The list of crawler runs for this crawler.

[Stop run](#) [View CloudWatch logs](#) [View run details](#)

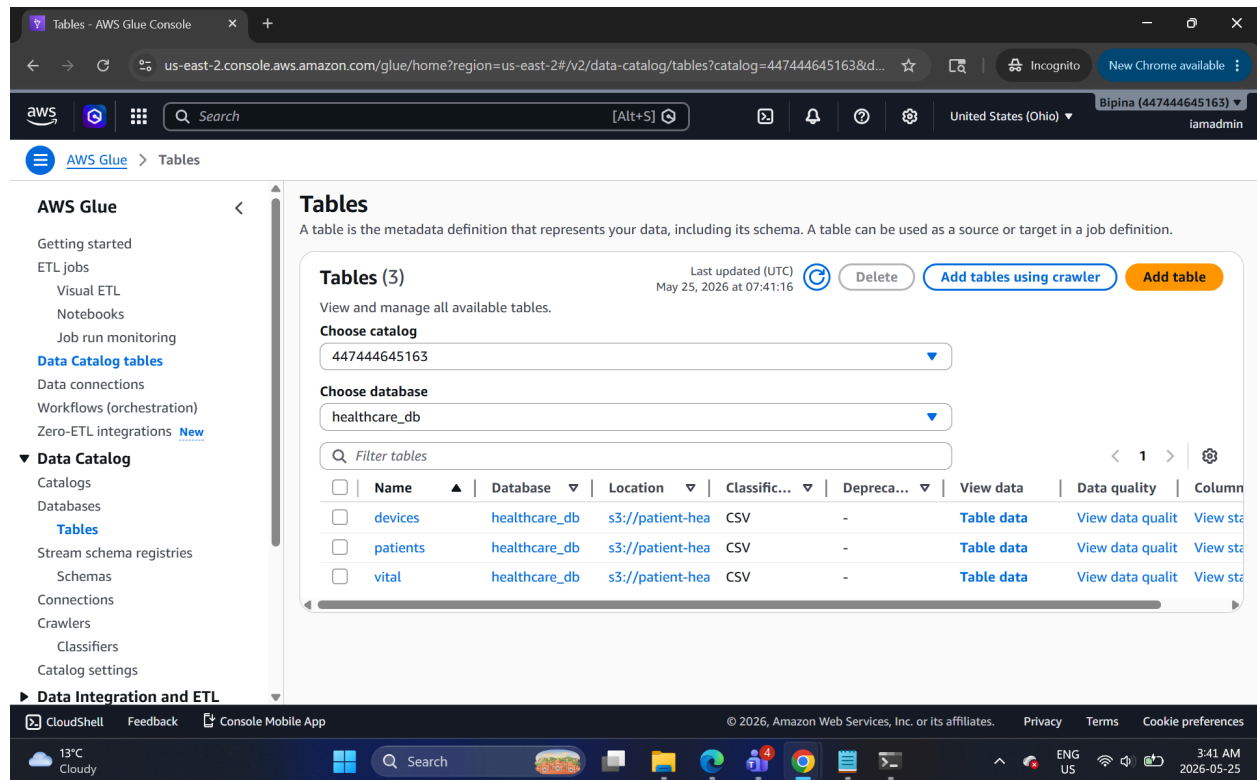
Started crawler successfully.

**Crawler runs (1)**  
The list of crawler runs for this crawler.

[Filter data](#) [Filter by a date and time range](#)

|                       | Start time (UTC)         | End time (UTC)           | Current/last duration | Status    | DPU hours |
|-----------------------|--------------------------|--------------------------|-----------------------|-----------|-----------|
| <input type="radio"/> | May 25, 2026 at 07:38:17 | May 25, 2026 at 07:39:03 | 46 s                  | Completed | 0.132     |

Crawler stores metadata in Glue Catalog successfully.



## 7. Created Glue ETL job

```
import sys
from awsglue.context import GlueContext
from awsglue.job import Job
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from pyspark.sql.functions import (
    col, when, to_date, hour, lit, coalesce
)

# Glue job setup
args = getResolvedOptions(sys.argv, ["JOB_NAME"])

sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session

job = Job(glueContext)
job.init(args["JOB_NAME"], args)

# S3 paths
```

```

bucket = "patient-health-monitoring"

vitals_path = f"s3://patient-health-monitoring-bipina-poudel/raw/vital/"
patients_path = f"s3://patient-health-monitoring-bipina-poudel/raw/patients/"
devices_path = f"s3://patient-health-monitoring-bipina-poudel/raw/devices/"

processed_path = f"s3://patient-health-monitoring-bipina-poudel/processed/reports/"
rejected_path = f"s3://patient-health-monitoring-bipina-poudel/rejected/"

# Step 1: Read raw CSV files
vitals_df = spark.read.option("header", True).csv(vitals_path)
patients_df = spark.read.option("header", True).csv(patients_path)
devices_df = spark.read.option("header", True).csv(devices_path)

# Step 2: Convert data types
vitals_df = vitals_df.withColumn("heart_rate", col("heart_rate").cast("int")) \
    .withColumn("oxygen_level", col("oxygen_level").cast("int")) \
    .withColumn("temperature", col("temperature").cast("double")) \
    .withColumn("timestamp", col("timestamp").cast("timestamp"))

# Step 3: Remove duplicate records
vitals_df = vitals_df.dropDuplicates(["patient_id", "timestamp"])

# Step 4: Separate rejected/bad records
rejected_df = vitals_df.filter(
    col("patient_id").isNull() |
    col("heart_rate").isNull() |
    col("oxygen_level").isNull() |
    col("temperature").isNull()
)

# Write rejected records
rejected_df.write.mode("overwrite").option("header", True).csv(rejected_path)

# Keep valid records only
valid_vitals_df = vitals_df.filter(
    col("patient_id").isNotNull() &
    col("heart_rate").isNotNull() &
    col("oxygen_level").isNotNull() &
    col("temperature").isNotNull()
)

# Step 5: Create transformation columns
valid_vitals_df = valid_vitals_df.withColumn(

```



```

    "health_status",
    when(col("oxygen_level") < 90, "Critical")
    .when(col("temperature") > 100, "Fever")
    .when(col("heart_rate") > 100, "High Risk")
    .otherwise("Normal")
)

valid_vitals_df = valid_vitals_df.withColumn(
    "alert_flag",
    when(col("health_status") != "Normal", lit("Y")).otherwise(lit("N"))
)

valid_vitals_df = valid_vitals_df.withColumn(
    "monitoring_date",
    to_date(col("timestamp"))
)

valid_vitals_df = valid_vitals_df.withColumn(
    "monitoring_hour",
    hour(col("timestamp"))
)

# Step 6: Join patient vitals with patient details and device logs
final_df = valid_vitals_df.join(
    patients_df,
    on="patient_id",
    how="left"
).join(
    devices_df,
    on="patient_id",
    how="left"
)

# Step 7: Select final output columns
final_df = final_df.select(
    col("patient_id"),
    col("name").alias("patient_name"),
    col("disease"),
    col("heart_rate"),
    col("oxygen_level"),
    col("temperature"),
    col("health_status"),
    col("alert_flag"),
    col("device_type"),

```

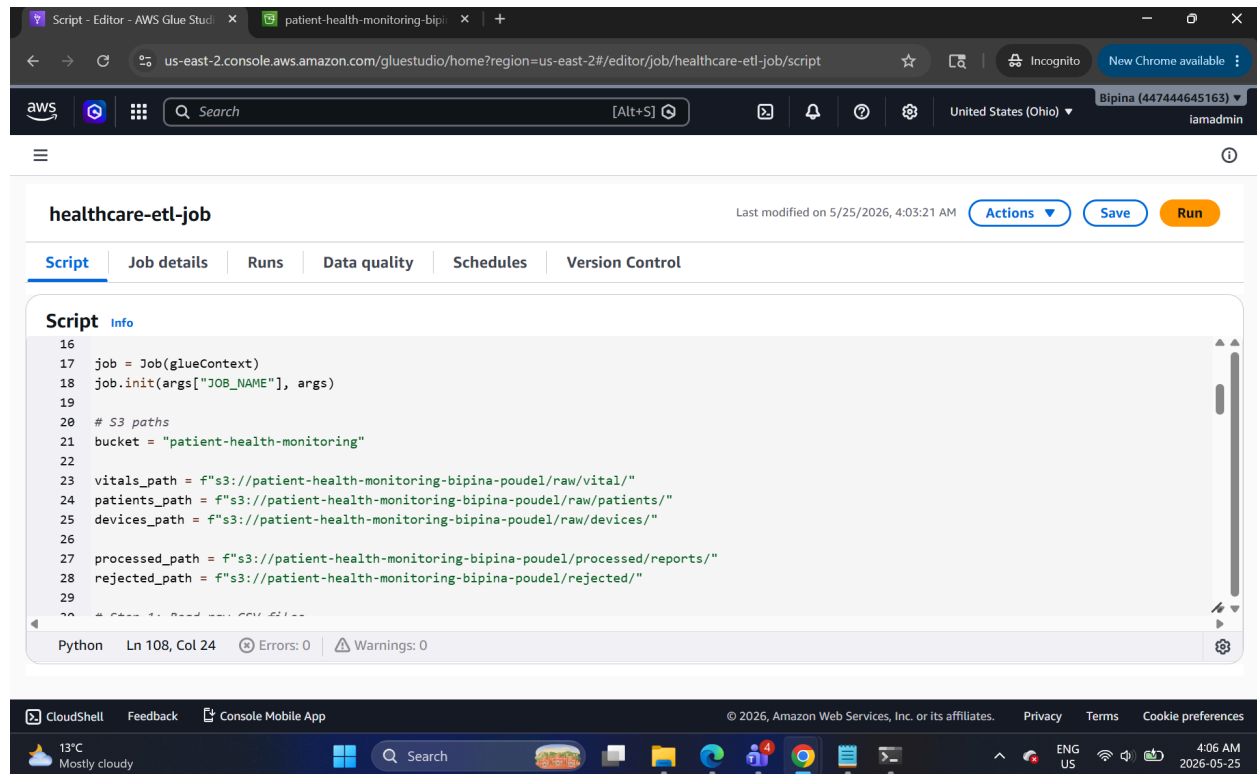
```

        col("monitoring_date"),
        col("monitoring_hour")
    )

# Step 8: Write final processed data as Parquet
final_df.write.mode("overwrite").parquet(processed_path)

job.commit()

```



Run ETL job successfully.

The screenshot shows the AWS Glue Studio interface for a job named 'healthcare-etl-job'. The job has been successfully completed. The 'Runs' tab is active, showing a single run with a status of 'Succeeded'. The run details show it started on May 25, 2026, at 04:03:46 and ended at 04:05:04, with a duration of 1 minute and 11 seconds. The job used 4 DPU capacity and G.1X worker type. The 'Logs' tab is also visible, showing the driver logs for the job.

| Run status | Retries | Start time (Local)  | End time (Local)    | Duration | Capacity (D... | Worker type | Glue vers |
|------------|---------|---------------------|---------------------|----------|----------------|-------------|-----------|
| Succeeded  | 0       | 05/25/2026 04:03:46 | 05/25/2026 04:05:04 | 1 m 11 s | 4 DPUs         | G.1X        | 5.1       |

Logs

The screenshot shows the AWS Glue Studio interface for the 'healthcare-etl-job' with the 'Logs' tab selected. The logs display the driver logs for the job, showing the execution of the DAG scheduler and the submission of tasks. The logs are organized into a table with columns for the log level, timestamp, and the log message.

| Log level | Timestamp                     | Log message                                                                                                 |
|-----------|-------------------------------|-------------------------------------------------------------------------------------------------------------|
| INFO      | 2026-05-25T08:04:31,877 22723 | org.apache.spark.scheduler.cluster.glue.ExecutorEventListener [spark-listener-group-shared] 60 Got exe      |
| INFO      | 2026-05-25T08:04:31,875 22721 | org.apache.spark.executor.ExecutorLogUrlHandler [dispatcher-CoarseGrainedScheduler] 60 Fail to renew e      |
| INFO      | 2026-05-25T08:04:31,874 22720 | org.apache.spark.scheduler.cluster.glue.JESSchedulerBackend\$JESAsSchedulerBackendEndpoint [dispatcher-Coar |
| INFO      | 2026-05-25T08:04:31,865 22711 | org.apache.spark.scheduler.TaskSchedulerImpl [dag-scheduler-event-loop] 60 Adding task set 0.0 with         |
| INFO      | 2026-05-25T08:04:31,864 22710 | org.apache.spark.scheduler.DAGScheduler [dag-scheduler-event-loop] 60 Submitting 1 missing tasks from       |
| INFO      | 2026-05-25T08:04:31,844 22690 | org.apache.spark.SparkContext [dag-scheduler-event-loop] 60 Created broadcast 3 from broadcast at D         |
| INFO      | 2026-05-25T08:04:31,843 22689 | org.apache.spark.storage.BlockManagerInfo [dispatcher-BlockManagerMaster] 60 Added broadcast_3_piece        |
| INFO      | 2026-05-25T08:04:31,841 22687 | org.apache.spark.storage.memory.MemoryStore [dag-scheduler-event-loop] 60 Block broadcast_3_piecer          |
| INFO      | 2026-05-25T08:04:31,822 22668 | org.apache.spark.storage.memory.MemoryStore [dag-scheduler-event-loop] 60 Block broadcast_3 store           |
| INFO      | 2026-05-25T08:04:31,713 22559 | org.apache.spark.scheduler.DAGScheduler [dag-scheduler-event-loop] 60 Submitting ResultStage 0 (MapPar      |
| INFO      | 2026-05-25T08:04:31,693 22539 | org.apache.spark.scheduler.DAGScheduler [dag-scheduler-event-loop] 60 Missing parents: List()               |
| INFO      | 2026-05-25T08:04:31,691 22537 | org.apache.spark.scheduler.DAGScheduler [dag-scheduler-event-loop] 60 Final stage: ResultStage 0 (csv       |
| INFO      | 2026-05-25T08:04:31,692 22538 | org.apache.spark.scheduler.DAGScheduler [dag-scheduler-event-loop] 60 Parents of final stage: List()        |
| INFO      | 2026-05-25T08:04:31,691 22537 | org.apache.spark.scheduler.DAGScheduler [dag-scheduler-event-loop] 60 Got job 0 (csv at NativeMethodAc      |

## 9. Checked final output

### Verifying output Parquet File in S3

The screenshot shows the Amazon S3 console interface. The left sidebar contains navigation options: Buckets, Files, Access management and security, and Storage management and insights. The main content area displays the 'reports/' bucket. Under the 'Objects (1)' section, there is a table with one object:

| Name                                                                              | Type    | Last modified                      | Size   | Storage class |
|-----------------------------------------------------------------------------------|---------|------------------------------------|--------|---------------|
| <a href="#">part-0000-be4acbef-2f31-4fde-b67d-957edd93adbfc000.snappy.parquet</a> | parquet | May 25, 2026, 04:04:52 (UTC-04:00) | 3.3 KB | Standard      |

Below the table, there is a search bar and pagination controls. The bottom of the screenshot shows the Windows taskbar with the time 4:10 AM on 2026-05-25.

## 10. Athena Validation

The screenshot shows the Amazon Athena console interface. The left sidebar contains navigation options: Tables (3) and Views (0). The main content area displays the 'Query editor' with a SQL query and its results.

**Query results**

Completed Time in queue: 77 ms Run time: 376 ms Data scanned: 0.25 KB

Results (5)

| # | patient_id | name         | age | gender | city   | disease       |
|---|------------|--------------|-----|--------|--------|---------------|
| 1 | P1001      | Rahul Sharma | 45  | Male   | Delhi  | Diabetes      |
| 2 | P1002      | Priya Verma  | 52  | Female | Mumbai | Hypertension  |
| 3 | P1003      | Amit Singh   | 60  | Male   | Pune   | Heart Disease |
| 4 | P1004      | Neha Kapoor  | 29  | Female | Delhi  | Asthma        |
| 5 | P1005      | Rohit Jain   | 48  | Male   | Jaipur | Diabetes      |

The bottom of the screenshot shows the Windows taskbar with the time 4:16 AM on 2026-05-25.

Runs - Editor - AWS Glue Studio | Query editor | Athena | us-east

us-east-2.console.aws.amazon.com/athena/home?region=us-east-2#/query-editor/history/2911a003-8a30-4c05-8e2c...

Amazon Athena > Query editor

Tables (3)

- devices
- patients
- vital

Views (0)

SQL Ln 1, Col 1

Run again Explain Cancel Clear Create

Reuse query results up to 60 minutes ago

Query results Query stats

Completed Time in queue: 102 ms Run time: 351 ms Data scanned: 0.20 KB

Results (4) Copy Download results CSV

Search rows

| # | col0      | col1       | col2         | col3     | col4                |
|---|-----------|------------|--------------|----------|---------------------|
| 1 | device_id | patient_id | device_type  | status   | last_sync           |
| 2 | D101      | P1001      | SmartWatch   | Active   | 2025-01-10 08:00:00 |
| 3 | D102      | P1002      | Oximeter     | Inactive | 2025-01-10 07:50:00 |
| 4 | D103      | P1003      | HeartMonitor | Active   | 2025-01-10 08:05:00 |

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

13°C Mostly cloudy

Search

ENG US 4:18 AM 2026-05-25

Runs - Editor - AWS Glue Studio | Query editor | Athena | us-east

us-east-2.console.aws.amazon.com/athena/home?region=us-east-2#/query-editor/history/0313adfb-226e-42fc-b5d9...

Amazon Athena > Query editor

Tables (3)

- devices
- patients
- vital

Views (0)

SQL Ln 1, Col 1

Run again Explain Cancel Clear Create

Reuse query results up to 60 minutes ago

Query results Query stats

Completed Time in queue: 74 ms Run time: 519 ms Data scanned: 0.33 KB

Results (6) Copy Download results CSV

Search rows

| # | patient_id | timestamp           | heart_rate | blood_pressure | oxygen_level | tem  |
|---|------------|---------------------|------------|----------------|--------------|------|
| 1 | P1001      | 2025-01-10 08:00:00 | 72         | 120/80         | 98           | 98.6 |
| 2 | P1002      | 2025-01-10 08:05:00 | 95         | 140/90         | 96           | 99.1 |
| 3 | P1003      | 2025-01-10 08:10:00 | 45         | 90/60          | 88           | 101. |
| 4 | P1004      | 2025-01-10 08:15:00 |            | 130/85         | 97           | 98.4 |
| 5 | P1005      | 2025-01-10 08:20:00 | 110        | 150/95         |              | 100. |

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

13°C Mostly cloudy

Search

ENG US 4:18 AM 2026-05-25